

TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

LABORATORY REPORT

OPERATING SYSTEMS LABORATORY 7

Three Small Programs: Processes, Redirection and a Pipe

Student: Lucian-Adrian Gavril

Group: FAF-241

Coordinator: univ. assist., Reițman Patricia

Chișinău, 2026

TABLE OF CONTENTS

ABSTRACT 3

INTRODUCTION 4

1 PART 1: FORK, EXEC AND WAIT 5

1.1 Observe 1 5

1.2 Observe 2 6

1.3 Observe 3 6

2 PART 2: A ZOMBIE 8

2.1 Observe 1 8

2.2 Observe 2 8

3 PART 3: REDIRECTION 10

3.1 Observe 1 10

3.2 Observe 2 10

3.3 Observe 3 10

4 PART 4: A PIPE 12

4.1 Observe 1 12

4.2 Observe 2 13

4.3 Observe 3 13

5 OPTIONAL THREE-STAGE PIPELINE 15

CONCLUSIONS 16

REFERENCES 17

APPENDICES 18

APPENDIX A – Program source 18

A.1 run.c 18

A.2 zombie.c 18

A.3 redirect.c 19

A.4 pipeline.c 20

A.5 pipeline3.c 21

ABSTRACT

This laboratory reconstructs shell command execution using Linux process and descriptor calls. Four programs demonstrate fork/exec/wait, a visible zombie, file redirection and a two-process pipeline. An optional three-process pipeline extends the same wiring. Observations under Ubuntu 24.04 WSL2 show the expected exit statuses, descriptor allocation and matching shell pipeline counts. A retained write-end reference explains the intentionally blocked consumer.

Keywords: processes, system calls, file descriptors, pipes, linux, xv6

INTRODUCTION

The required experiments follow the laboratory handout[1]; the process and descriptor model follows the course lecture[2]. The supplied source is reproduced in Appendix A. A successful execution is evidence for that captured environment, while the concept figures explain the general mechanism.

The UNIX process interface lets a shell compose small programs without changing their implementations. Process calls control which program runs and when its completion is collected; file descriptors select where the program reads and writes. These mechanisms explain redirection, pipelines and communication between processes. This lab reconstructs shell command execution in C. Four programs exercise process creation and replacement, child reaping, output redirection and a two-stage pipeline. A three-stage pipeline extends the same descriptor-wiring mechanism.

Measured environment: Ubuntu 24.04 under WSL2, kernel `6.18.33.2-microsoft-standard-WSL2`, GCC `13.3.0`. The 19 October 2026 capture session compiled all five programs with `gcc -Wall -Wextra`.

1 PART 1: FORK, EXEC AND WAIT

Figure 1.1 distinguishes a new process from a replacement program. The amber interval represents an exited child retained until its parent reaps it; the child executes no instructions in that interval.

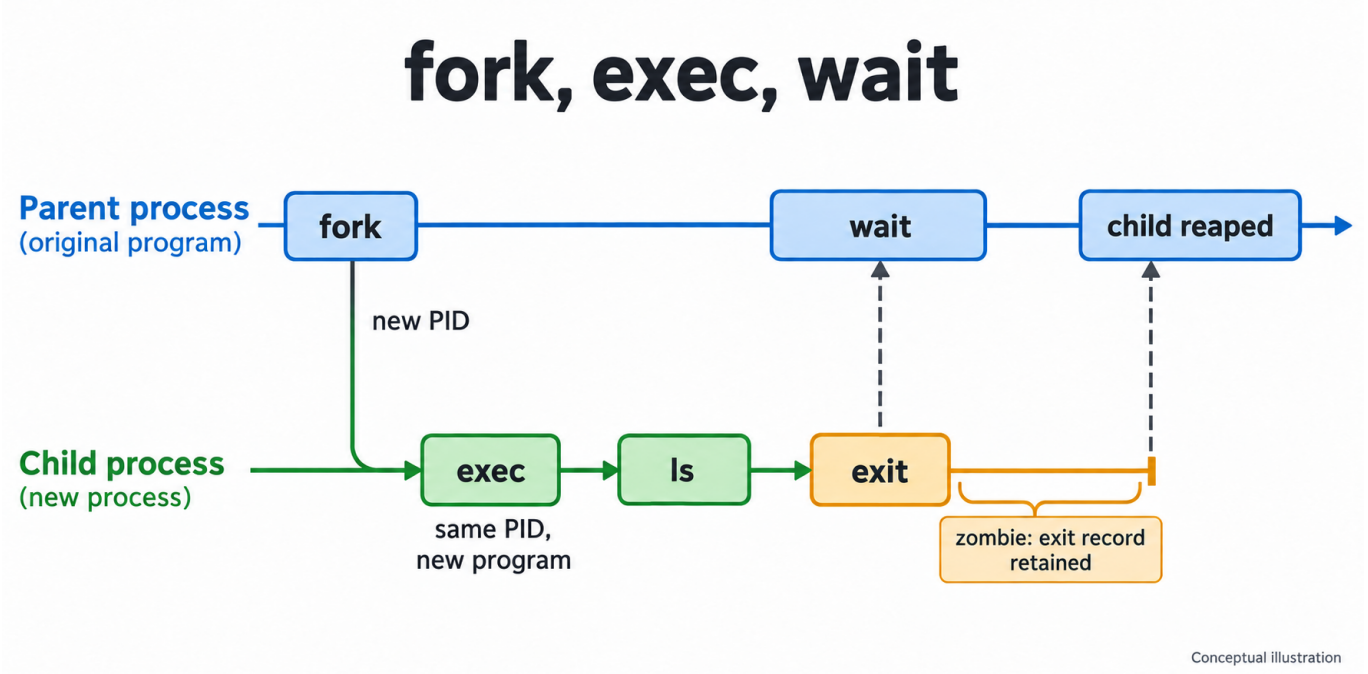


Figure 1.1 – Process lifetime and the retained exit record. ImageGen concept illustration, 23 October 2026; scheduling is schematic.

The related xv6 implementation can be compared in the course source snapshot[3]. A program is stored instructions; a process is a running instance with an address space, registers and open descriptors. A successful `fork` creates a child and returns twice: zero in the child and the child PID in the parent. Their relative scheduling is not fixed. In this program the parent waits, which puts its completion message after the child has finished.

The child calls `execlp` to replace its program with `ls`. Successful replacement keeps the PID, but changes the program image. It never returns to the old child code. The parent remains in `run` and collects the child's status with `waitpid`. Linux encodes the termination information in the status word; `WEXITSTATUS` is meaningful for normal termination. The supplied example assumes that path; general code also checks `WIFEXITED`.

1.1 Observe 1

Measured: The canonical `ls` child exited with status `0`; the temporary `date` child also exited with status `0`. Neither successful execution printed the `perror("exec")` message. With `nosuchprog`, the child printed `exec: No such file or directory` and the parent reported exit status `127`.

Explanation: A successful `execlp` replaces the child's program image and does not return to the

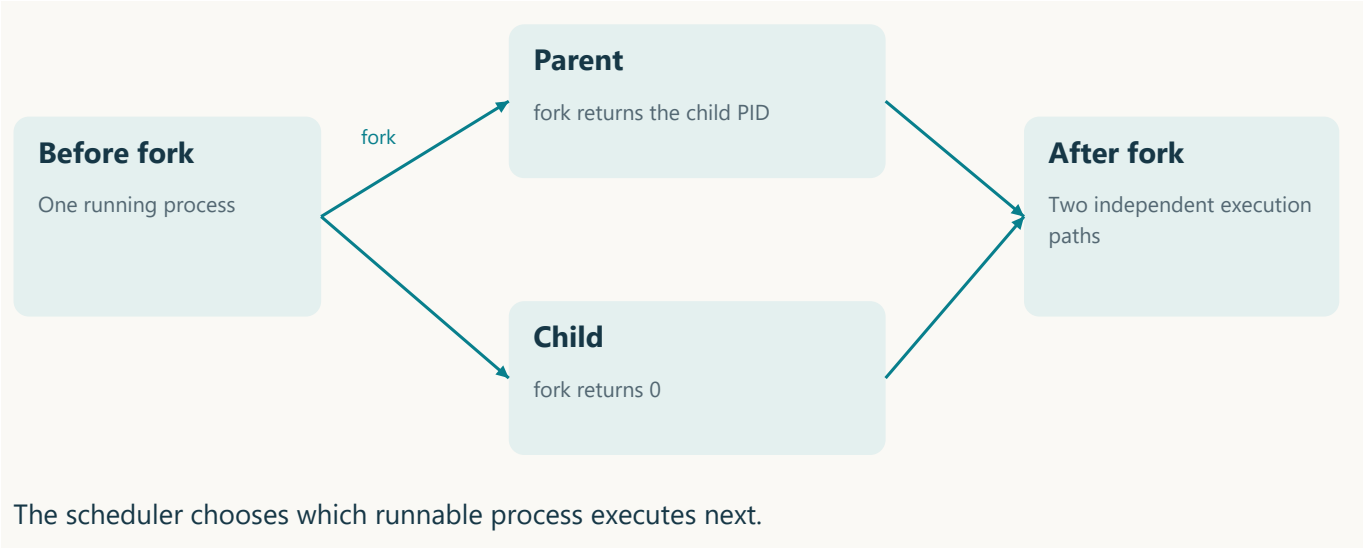


Figure 1.2 – fork creates two execution paths. The parent and child distinguish their paths by the return value.

old code, so execution never reaches `perror`. When the requested program is missing, `execlp` returns with an error; the next line reports it, and the child exits with `127`.

1.2 Observe 2

Measured: The parent printed PID `1795`; the child printed its parent PID as `1795` and its own PID as `1796`.

Explanation: `getppid()` returns the PID of the process that created this child. The parent stays alive in `waitpid`, so its identity has not changed before the child prints.

1.3 Observe 3

Measured: The parent reported status `0` for `ls` and `127` for `nosuchprog`. The `date` variant reported status `0` as well.

Explanation: The `ls` program exited successfully with status `0`. The failed-`exec` branch explicitly calls `exit(127)`. `waitpid` collects that child's status for the parent.

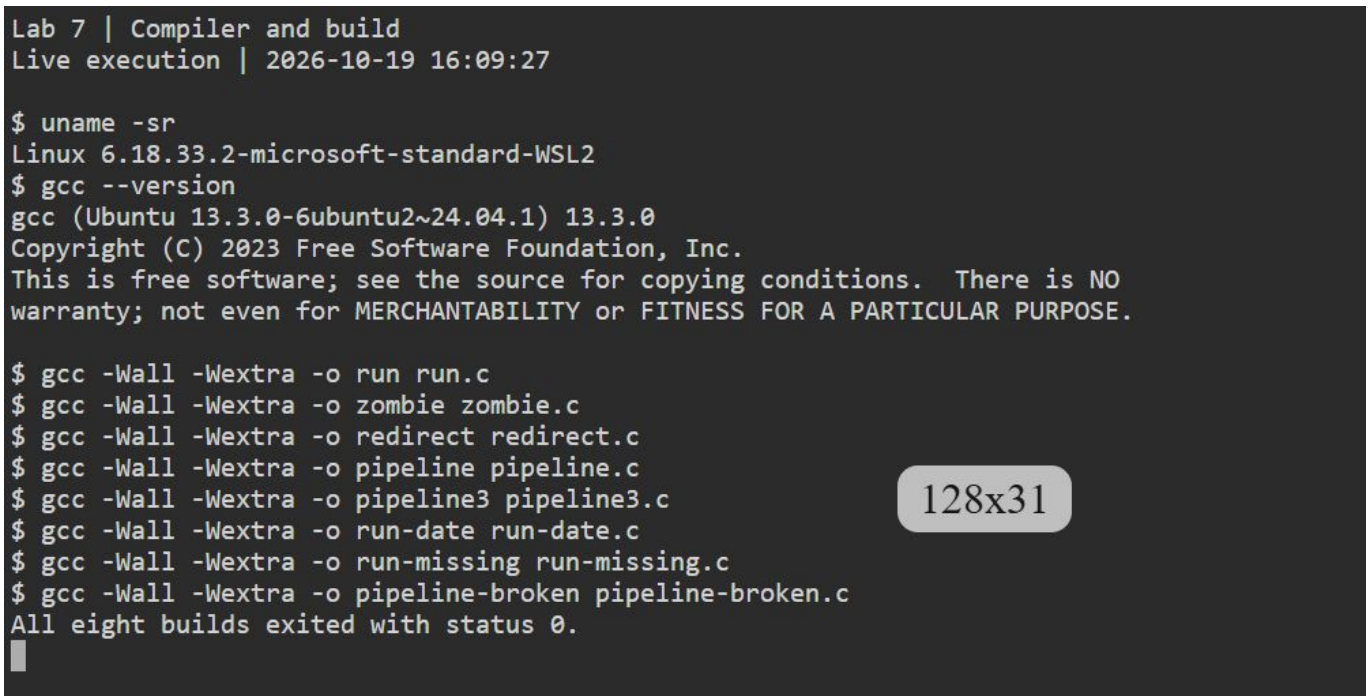


Figure 1.3 – Live Linux console: GCC version and successful builds with -Wall -Wextra. Live ttyd terminal capture, 19 October 2026. Only blank margins are cropped in this report; program output is unchanged.

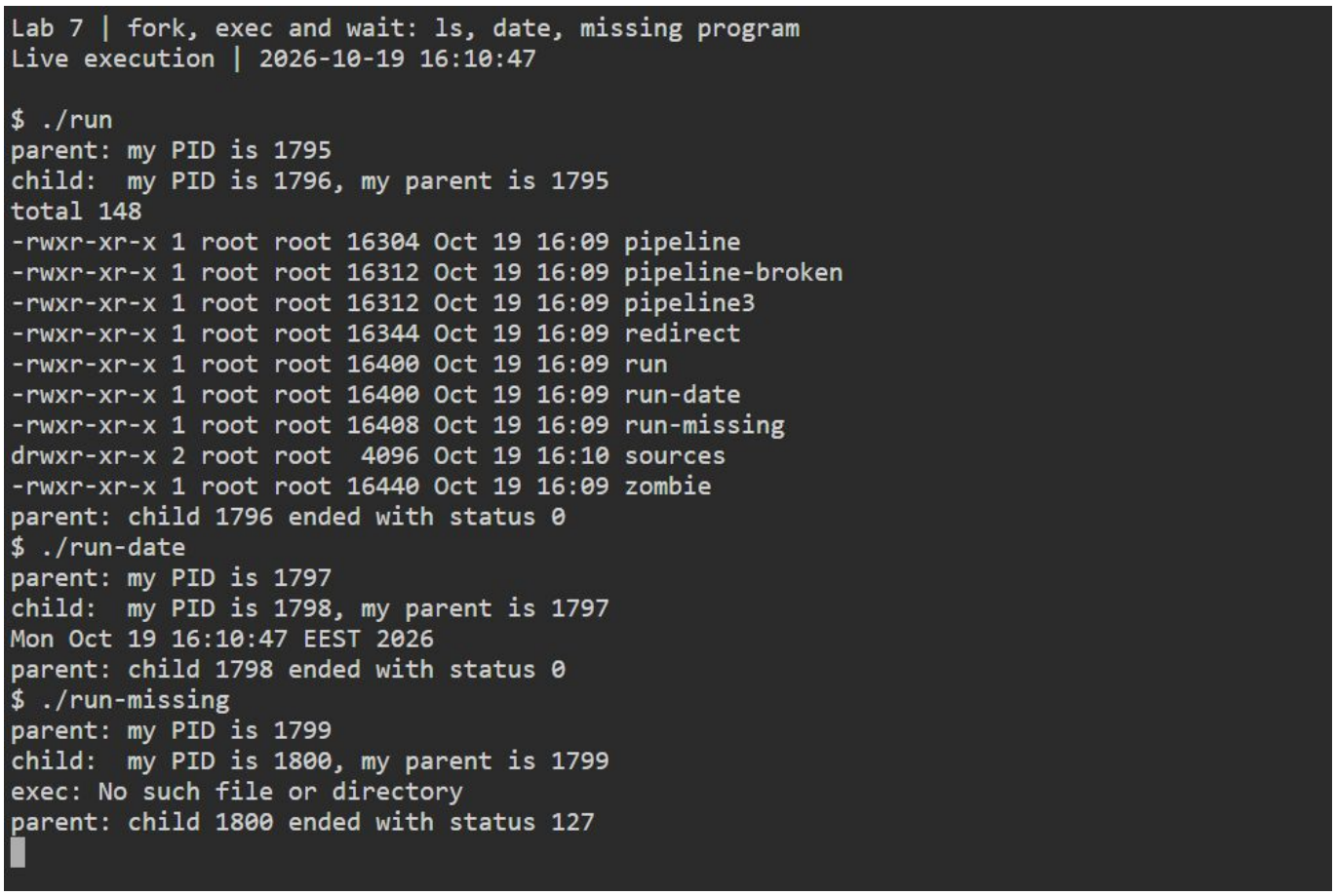


Figure 1.4 – Live Linux console: the ls and date replacements succeed; the missing-command branch reports exit status 127. Live ttyd terminal capture, 19 October 2026. Only blank margins are cropped in this report; program output is unchanged.

2 PART 2: A ZOMBIE

When a child exits, it stops executing and releases its ordinary execution resources. The kernel retains the PID and completion information until its parent collects them. This exited, unreaped state is called a zombie. It differs from a running orphan whose parent has exited. Delaying the parent's wait exposes the zombie; sleeping does not itself reap the child.

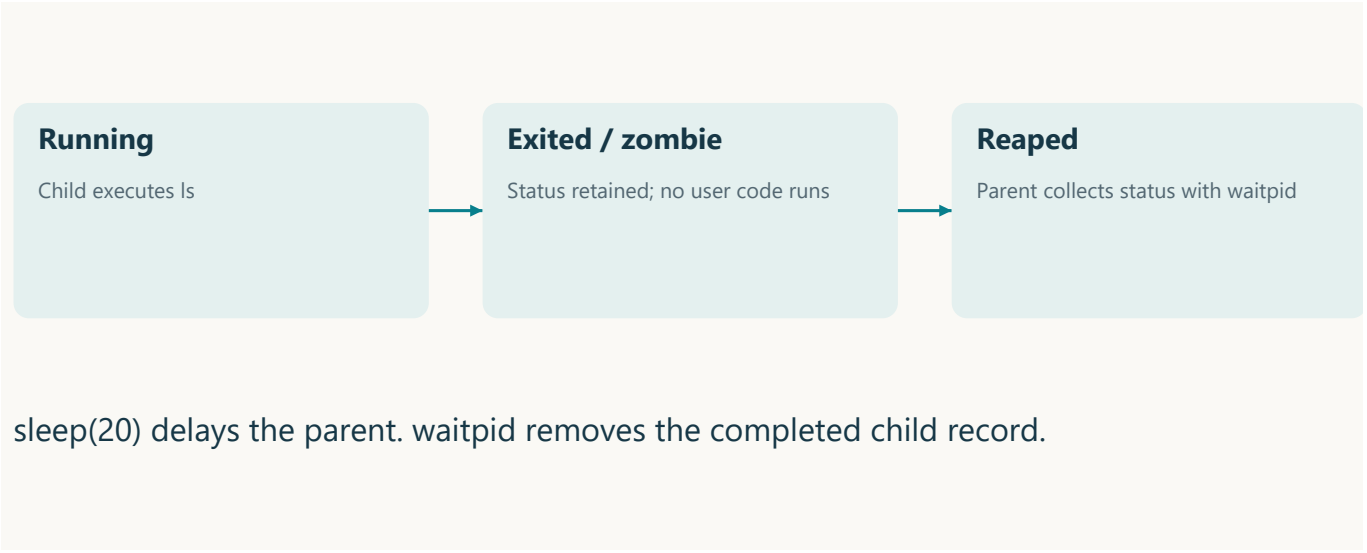


Figure 2.1 – An exited child retains an exit record until its parent reaps it.

2.1 Observe 1

Measured: While parent PID `1893` slept, child PID `1894` had PPID `1893` and `STAT` value `Z+`; `ps` showed `[ls] <defunct>`. The `+` is an additional process-group marker; `Z` is the zombie state asked about in the brief.

Explanation: `Z` means the child has exited but its parent has not yet collected its exit status. It is not still executing `ls`.

2.2 Observe 2

Measured: After the 20-second sleep and `waitpid`, `ps` showed no row for parent PID `1893` or its child.

Explanation: The `waitpid(pid, &status, 0)` at `zombie.c:25` collects the child's remaining exit record. `sleep(20)` at line 24 only creates time to observe that record.

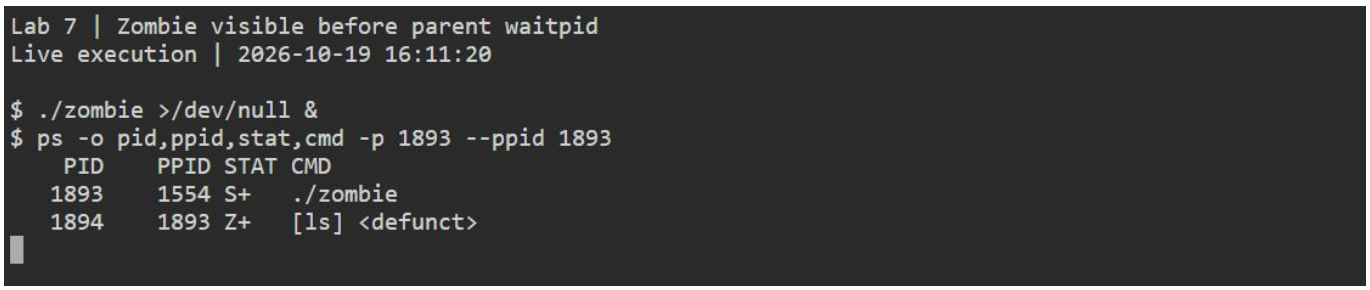


Figure 2.2 – Before the parent waitpid completes, ps shows the child in zombie state Z. Live ttyd terminal capture, 19 October 2026. Only blank margins are cropped in this report; program output is unchanged.

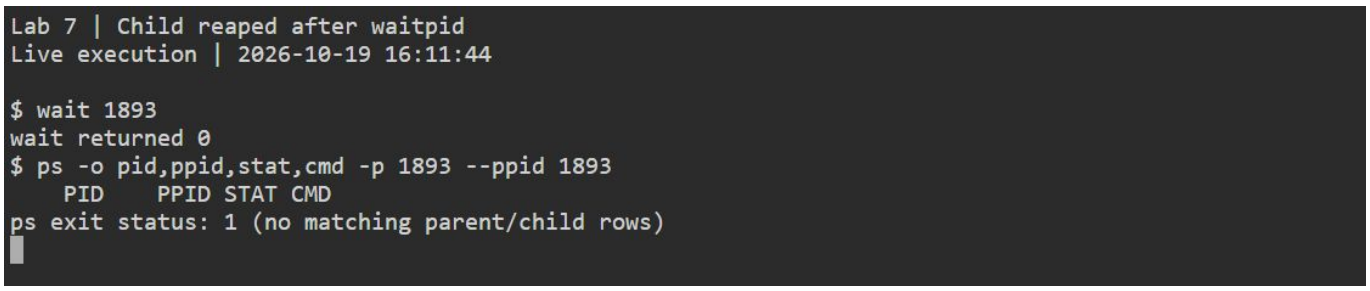


Figure 2.3 – After waitpid and parent completion, ps no longer finds either of the recorded process rows. Live ttyd terminal capture, 19 October 2026. Only blank margins are cropped in this report; program output is unchanged.

3 PART 3: REDIRECTION

A file descriptor is an index in a process's descriptor table, not the file itself. By convention, 0 is standard input, 1 is standard output and 2 is standard error. After `close(1)`, the lowest free index is 1. Opening `out.txt` assigns that index to the file. The new program inherits the descriptor connection across `exec`, so an unchanged `ls` writes into the file. These changes occur in the child and do not redirect the parent's completion message.

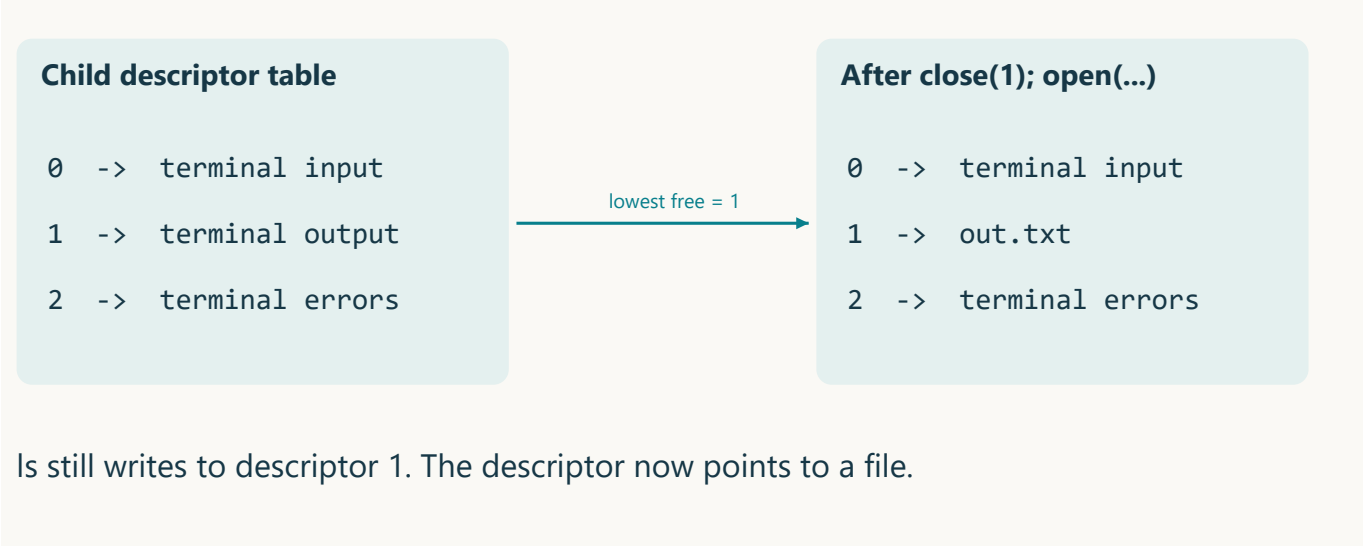


Figure 3.1 – The child changes descriptor 1 from the terminal to out.txt; descriptor 2 continues to carry diagnostics.

3.1 Observe 1

Measured: `open` reported descriptor `1`.

Explanation: `close(1)` frees standard output in the child. `open` allocates the lowest available descriptor, which is now `1`.

3.2 Observe 2

Measured: The directory listing was in `out.txt`; the terminal showed the open diagnostic and the parent's completion line.

Explanation: The child uses descriptor 2 for the diagnostic, so it remains connected to the terminal. After `open`, descriptor `1` refers to `out.txt`. `ls` writes to standard output without knowing that the destination changed.

3.3 Observe 3

To redirect errors like `2> err.txt`, close descriptor 2 before calling `open`. Then the file receives the lowest free descriptor, 2, which is standard error.

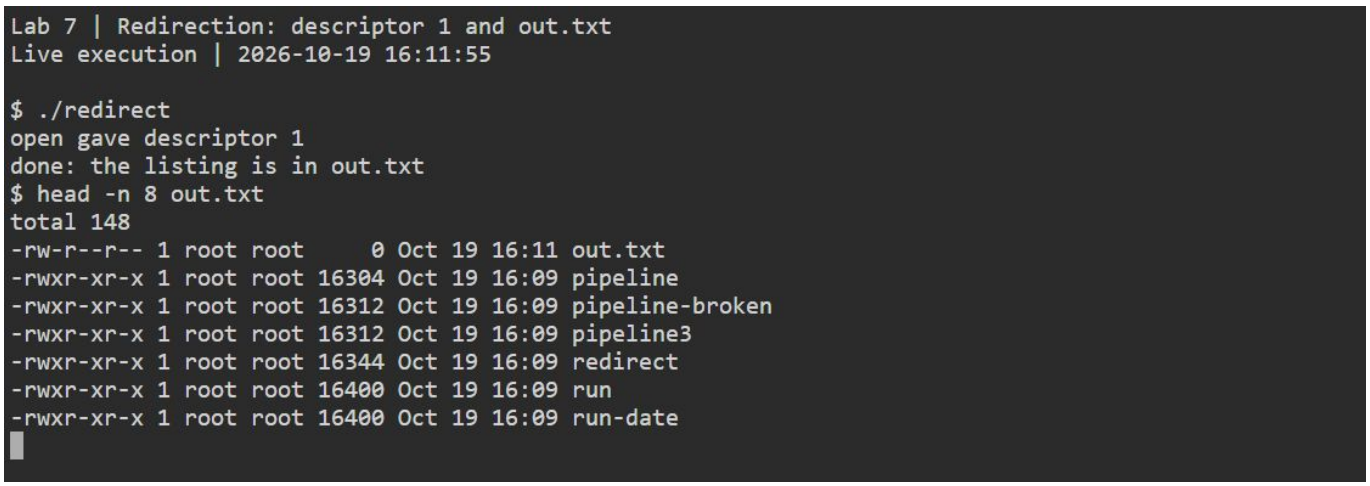


Figure 3.2 – open returns descriptor 1; head shows the directory listing stored in out.txt. Live ttyd terminal capture, 19 October 2026. Only blank margins are cropped in this report; program output is unchanged.

4 PART 4: A PIPE

Figure 4.1 shows why closing unused endpoints is part of the algorithm. The two-direction xv6 exchange applies this rule independently to each pipe.

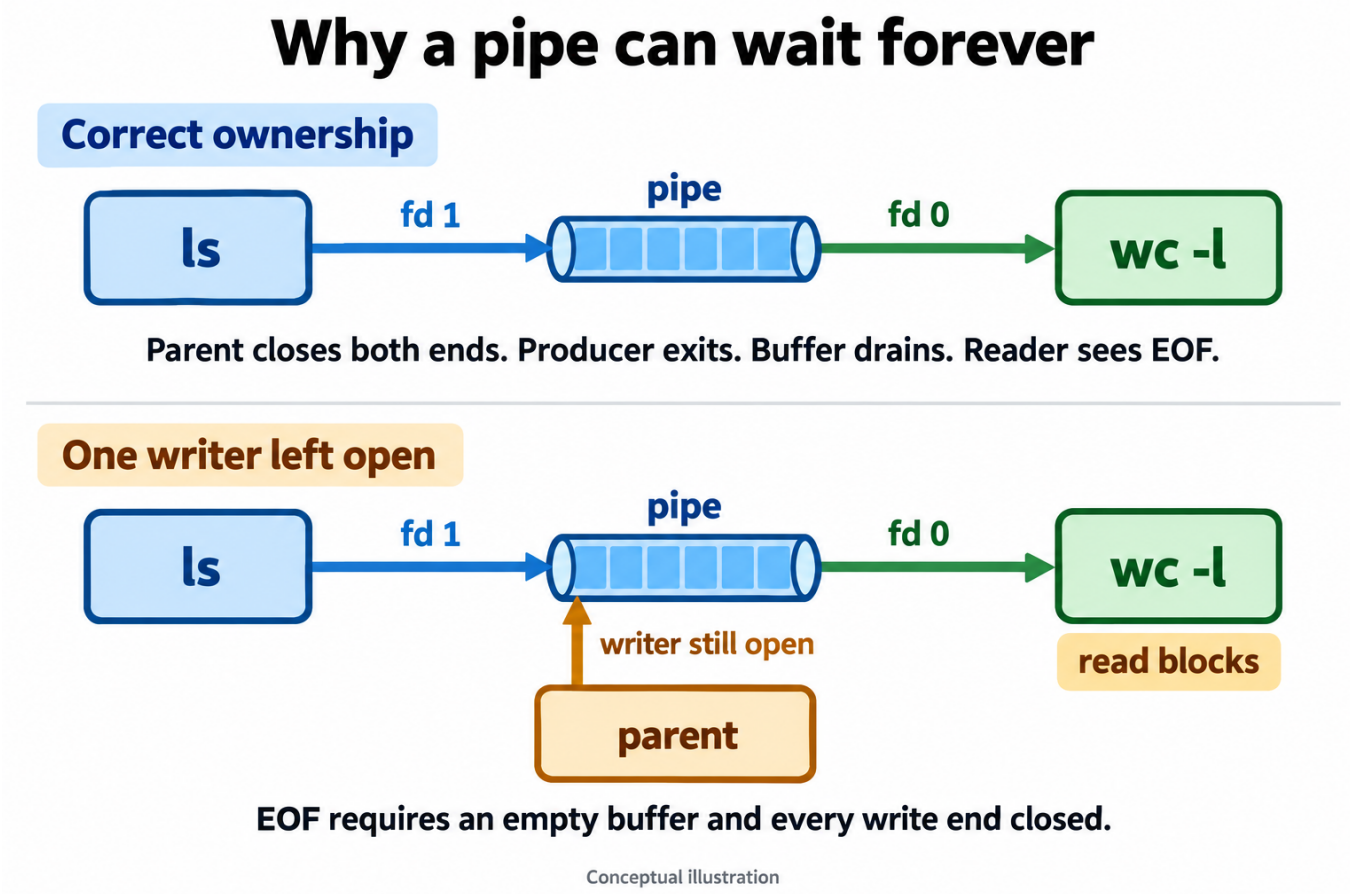


Figure 4.1 – Writer ownership determines end-of-file. ImageGen concept illustration, 23 October 2026; not measured output.

`pipe(p)` creates a byte stream with a read end `p[0]` and a write end `p[1]`. Creating it before either fork lets both children inherit references to the same pipe. The producer duplicates the writer onto descriptor 1; the consumer duplicates the reader onto descriptor 0. The original endpoints are then closed in both children, and the parent closes both copies. `wc -l` counts newline bytes, not structured filesystem entries.

A reader reaches end-of-file only when the buffer is empty and every reference to the write end has closed. If the parent retains its writer, an empty pipe can still receive data, so `wc` waits. The parent may then wait for `wc`, exposing the descriptor-lifetime mistake.

4.1 Observe 1

Measured: `./pipeline` printed `172` and the shell command `ls /etc | wc -l` printed `172` in this run.

Explanation: The first child sends the `ls /etc` listing through the pipe. The second reads that byte

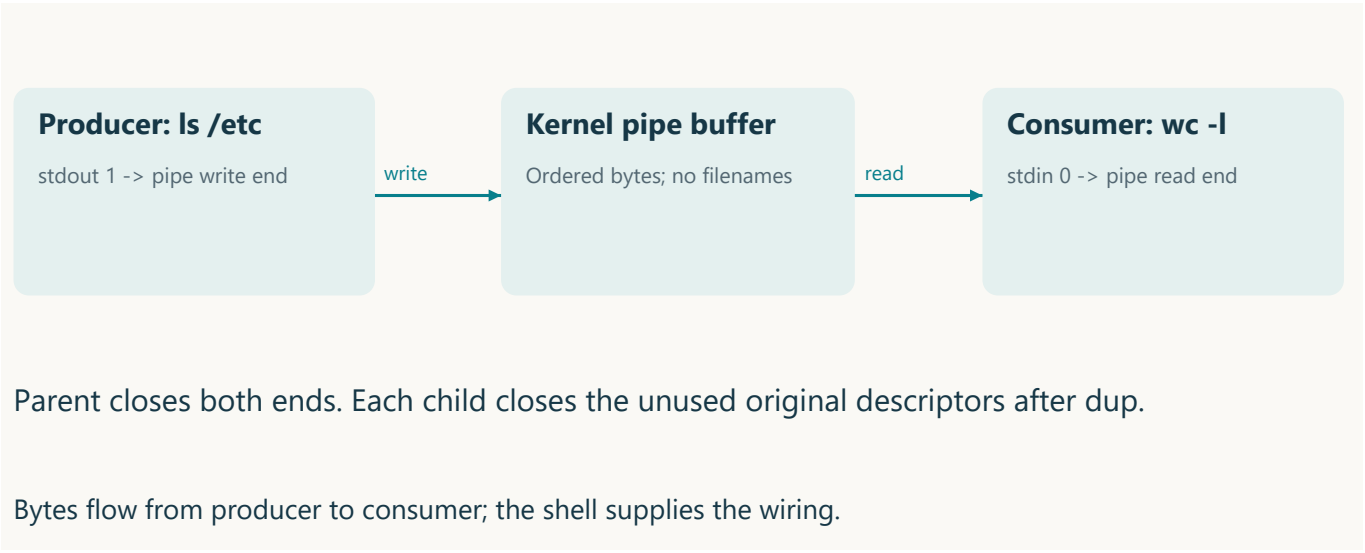


Figure 4.2 – Two child processes connect standard output and standard input through one kernel pipe buffer.

stream and counts newline characters with `wc -l`. With the same `/etc` contents, the hand-built pipeline and shell pipeline count the same lines.

4.2 Observe 2

Measured: The intentionally broken variant's parent remained blocked after the bounded 0.5-second wait. The `ps` snapshot showed `wc -l` PID 2007 still running with PPID 2005, the pipeline parent. The capture runner then sent SIGTERM to the isolated process group 2005 and waited for the pipeline parent. `ls` had finished.

Explanation: `wc` waits for end-of-file. The parent retained a copy of the pipe's write end, so the kernel could not report that all writers were gone even after `ls` exited.

4.3 Observe 3

`wc` is waiting for end-of-file on its standard input. End-of-file reaches the reader only after every open reference to the pipe's write end has been closed and the buffered bytes have been consumed.

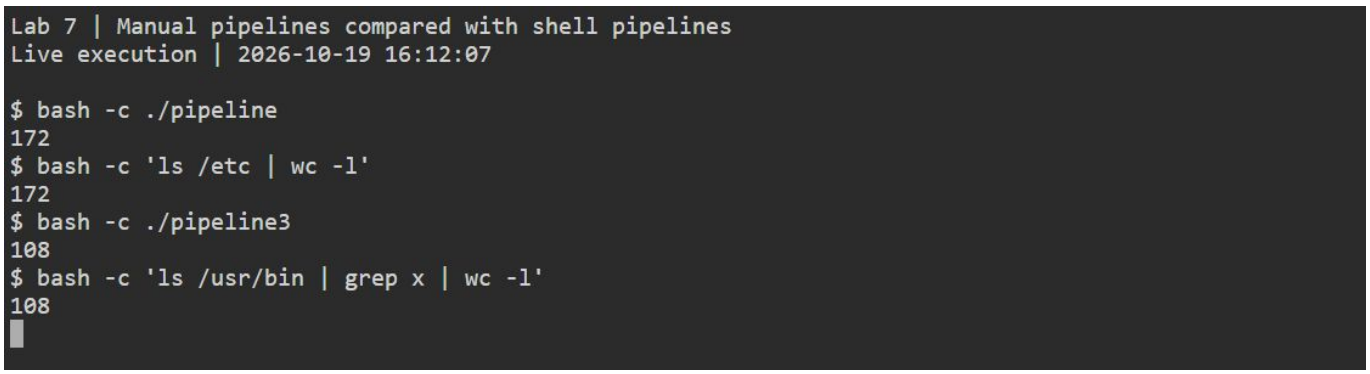


Figure 4.3 – The manual two-stage and optional three-stage pipelines match their shell equivalents. Live ttyd terminal capture, 19 October 2026. Only blank margins are cropped in this report; program output is unchanged.

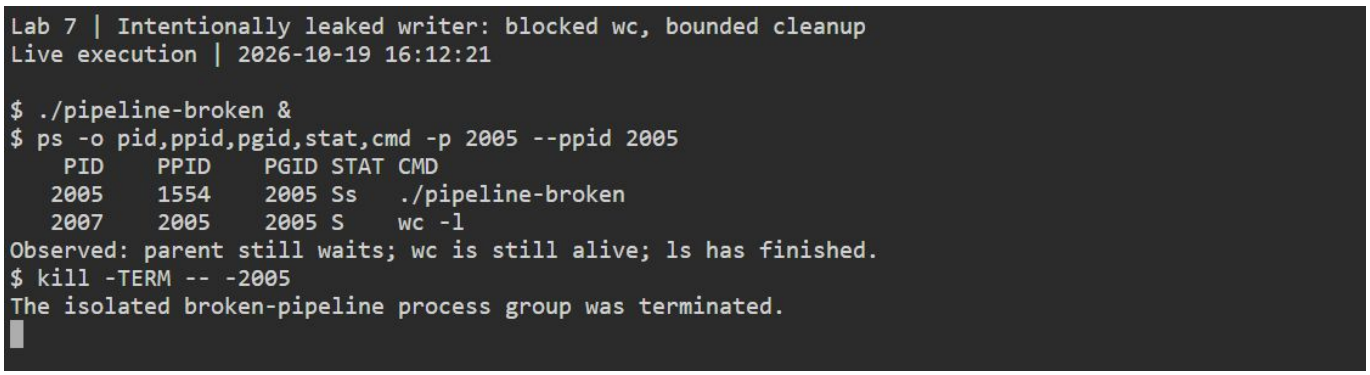


Figure 4.4 – With the parent writer deliberately retained, wc remains alive while the parent waits; the isolated group is then stopped. Live ttyd terminal capture, 19 October 2026. Only blank margins are cropped in this report; program output is unchanged.

5 OPTIONAL THREE-STAGE PIPELINE

The optional extension uses two pipes and three children. The first pipe carries the directory listing to `grep x`; the second carries matching lines to `wc -l`. Each child keeps only its own input and output connections. Closing every unused endpoint allows EOF to propagate through both stages.

Measured: `pipeline3` printed `108` and `ls /usr/bin | grep x | wc -l` printed `108` in this run.

Explanation: `ls` writes names to the first pipe, `grep x` reads those names and writes matching lines to the second pipe, and `wc -l` counts them. All three children and the parent close unused descriptors so each reader can receive end-of-file.

CONCLUSIONS

The Linux results connect process lifetime to descriptor lifetime. Successful exec replaces the program without creating a new PID; waitpid reaps an exited child. Redirection works by changing descriptor 1 before exec. A pipeline completes when the producer finishes and every writer reference closes, allowing the consumer to observe EOF. Both manual pipelines matched their shell equivalents in the recorded environment.

REFERENCES

1. Operating Systems, FCIM/FAF, UTM. *Laboratory 7 handout*, academic year 2026–2027. Accessed 19 October 2026. `https://else.fcim.utm.md/mod/resource/view.php?id=72238`.

2. Operating Systems. *Processes, Descriptors, Pipes*, Session 4. Accessed 19 October 2026. `https://else.fcim.utm.md/mod/resource/view.php?id=72236`.

3. MIT. *xv6-riscv source code*. Course source inspected 19 October 2026. Local course snapshot: `os/lab2`; `kernel/proc.c`, `kernel/exec.c`, `kernel/pipe.c` and `user/sh.c`.

APPENDICES

APPENDIX A – Program source

The following listings are the submitted programs. They accompany the observations in the main report.

A.1 run.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* the three steps of the shell for the command:   ls -l   */
int main(void)
{
    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);

    pid_t pid = fork();                          /* step 1: copy myself */
    if (pid < 0) { perror("fork"); return 1; }

    if (pid == 0) {                              /* only the child runs this */
        printf("child:  my PID is %d, my parent is %d\n", getpid(), getppid());
        fflush(stdout);
        execlp("ls", "ls", "-l", (char *)NULL);  /* step 2: become ls */
        perror("exec");                          /* runs only if exec failed */
        exit(127);
    }

    int status;
    waitpid(pid, &status, 0);                    /* step 3: wait for the child */
    printf("parent: child %d ended with status %d\n", pid, WEXITSTATUS(status));
    return 0;
}
```

A.2 zombie.c

```c
#include <stdio.h>
#include <stdlib.h>
```

```c
#include <unistd.h>
#include <sys/wait.h>

/* the three steps of the shell for the command:   ls -l   */
int main(void)
{
    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);

    pid_t pid = fork();                          /* step 1: copy myself */
    if (pid < 0) { perror("fork"); return 1; }

    if (pid == 0) {                              /* only the child runs this */
        printf("child:  my PID is %d, my parent is %d\n", getpid(), getppid());
        fflush(stdout);
        execlp("ls", "ls", "-l", (char *)NULL);  /* step 2: become ls */
        perror("exec");                          /* runs only if exec failed */
        exit(127);
    }

    int status;
    sleep(20);
    waitpid(pid, &status, 0);                    /* step 3: wait for the child */
    printf("parent: child %d ended with status %d\n", pid, WEXITSTATUS(status));
    return 0;
}
```

A.3 redirect.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

/* does the same as the shell command:   ls -l > out.txt   */
int main(void)
{
    pid_t pid = fork();
    if (pid == 0) {
        close(1);                                /* descriptor 1 is now free */
```

```
        int fd = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
        if (fd < 0) { perror("open"); exit(1); }
        fprintf(stderr, "open gave descriptor %d\n", fd);
        execlp("ls", "ls", "-l", (char *)NULL);  /* ls writes to 1, as always */
        perror("exec");
        exit(127);
    }
    waitpid(pid, NULL, 0);
    printf("done: the listing is in out.txt\n");
    return 0;
}
```

A.4 pipeline.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* does the same as the shell command:   ls /etc | wc -l   */
int main(void)
{
    int p[2];
    if (pipe(p) < 0) { perror("pipe"); return 1; }

    if (fork() == 0) {                          /* ls: its 1 becomes p[1] */
        close(1);
        dup(p[1]);                              /* smallest free number: 1 */
        close(p[0]);
        close(p[1]);
        execlp("ls", "ls", "/etc", (char *)NULL);
        perror("exec ls");
        exit(127);
    }

    if (fork() == 0) {                          /* wc: its 0 becomes p[0] */
        close(0);
        dup(p[0]);                              /* smallest free number: 0 */
        close(p[0]);
        close(p[1]);
        execlp("wc", "wc", "-l", (char *)NULL);
```

```
        perror("exec wc");
        exit(127);
    }

    close(p[0]);                                    /* the parent uses neither end */
    close(p[1]);
    wait(NULL);
    wait(NULL);
    return 0;
}
```

A.5 pipeline3.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* does the same as the shell command: ls /usr/bin | grep x | wc -l */
int main(void)
{
    int first[2];
    int second[2];
    if (pipe(first) < 0) { perror("pipe first"); return 1; }
    if (pipe(second) < 0) {
        perror("pipe second");
        close(first[0]);
        close(first[1]);
        return 1;
    }

    pid_t ls_pid = fork();
    if (ls_pid < 0) {
        perror("fork ls");
        close(first[0]);
        close(first[1]);
        close(second[0]);
        close(second[1]);
        return 1;
    }
    if (ls_pid == 0) {
```

```
        close(STDOUT_FILENO);
        if (dup(first[1]) != STDOUT_FILENO) {
            perror("dup ls");
            exit(1);
        }
        close(first[0]);
        close(first[1]);
        close(second[0]);
        close(second[1]);
        execlp("ls", "ls", "/usr/bin", (char *)NULL);
        perror("exec ls");
        exit(127);
    }

    pid_t grep_pid = fork();
    if (grep_pid < 0) {
        perror("fork grep");
        close(first[0]);
        close(first[1]);
        close(second[0]);
        close(second[1]);
        waitpid(ls_pid, NULL, 0);
        return 1;
    }
    if (grep_pid == 0) {
        close(STDIN_FILENO);
        if (dup(first[0]) != STDIN_FILENO) {
            perror("dup grep input");
            exit(1);
        }
        close(STDOUT_FILENO);
        if (dup(second[1]) != STDOUT_FILENO) {
            perror("dup grep output");
            exit(1);
        }
        close(first[0]);
        close(first[1]);
        close(second[0]);
        close(second[1]);
        execlp("grep", "grep", "x", (char *)NULL);
```

```
        perror("exec grep");
        exit(127);
    }

    pid_t wc_pid = fork();
    if (wc_pid < 0) {
        perror("fork wc");
        close(first[0]);
        close(first[1]);
        close(second[0]);
        close(second[1]);
        waitpid(ls_pid, NULL, 0);
        waitpid(grep_pid, NULL, 0);
        return 1;
    }
    if (wc_pid == 0) {
        close(STDIN_FILENO);
        if (dup(second[0]) != STDIN_FILENO) {
            perror("dup wc");
            exit(1);
        }
        close(first[0]);
        close(first[1]);
        close(second[0]);
        close(second[1]);
        execlp("wc", "wc", "-l", (char *)NULL);
        perror("exec wc");
        exit(127);
    }

    close(first[0]);
    close(first[1]);
    close(second[0]);
    close(second[1]);
    waitpid(ls_pid, NULL, 0);
    waitpid(grep_pid, NULL, 0);
    waitpid(wc_pid, NULL, 0);
    return 0;
}
```